

Project Overview

MERN stack | Banking simulation | Four user roles

Student: Athukorala A A C P | Student ID: IT22139962

Purpose

This full-stack web application simulates everyday banking operations, customer support, and user administration. Customers manage demo accounts, transfer simulated funds, track transactions, and submit complaints. It is an academic demonstration and does not connect to real banking networks or move real money.

Users and responsibilities

Customer: Register and sign in, view balances, transfer funds, search transaction history, submit complaints to a selected admin, and read notifications.

Customer Service Officer: Work with assigned complaints, update statuses, add staff notes, and escalate complex issues.

Bank Manager: Review escalated complaints, add manager notes, and view complaint reports and statistics.

System Admin: Manage users and staff, change roles and account access, delete users, and view or delete complaints assigned to their account.

Main modules

Authentication and role access; account information; fund transfers and transaction tracking; complaint handling; notifications; and system administration.

Recent additions

Complaint submission includes an active-admin dropdown. The selected admin sees the complaint on their dashboard and can delete it after confirmation. Admins can also delete users through the user details page; self-deletion is blocked.

Architecture & Data

How the frontend, backend, and database work together

Technology stack

Frontend: React and Vite build the interface. React Router handles navigation, Bootstrap provides UI styling, Axios sends API requests, and Context API shares authentication state.

Backend: Node.js and Express provide REST APIs. JWT identifies signed-in users, bcryptjs hashes passwords, and express-validator checks registration inputs.

Database: MongoDB stores application data. Mongoose defines schemas, relationships, and field validation.

Request flow

React page -> Axios request -> Express route -> Authentication and role checks -> Controller -> Mongoose model -> MongoDB

The backend returns a JSON response. The frontend uses that response to update the screen or show a success or error message.

Five core data models

User: Names, email, password hash, role, and active/inactive status.

Account: Account number, owner, account type, balance, and status.

Transaction: Sender and receiver accounts, amount, reference number, status, description, and timestamps.

Complaint: Customer, title, description, category, assigned admin/officer, status, staff/manager notes, and lifecycle timestamps.

Notification: Recipient, title, message, notification type, read status, and timestamps.

Relationships

Accounts reference users. Each transfer references two accounts. Complaints reference a customer and assigned staff. Notifications reference the user who receives them.

Structure & Workflows

A guide to the code and the main user journeys

Frontend folders

client/src/pages/: Screens grouped by customer, officer, manager, admin, and authentication.
components/: Shared layouts and UI. context/: Authentication state. services/api.js: API calls. App.jsx: Page routes and role restrictions.

Backend folders

server/routes/: API endpoints. controllers/: Business logic. models/: Database schemas.
middleware/: Authentication, roles, and errors. utils/: Currency and identifier helpers.
tests/: Backend tests. seed/: Demo data. server.js: Server startup and database connection.

Fund transfer workflow

1. Customer enters the receiver account and transfer amount.
 2. Backend checks identity, account status, amount precision, and available balance.
 3. A database transaction updates both balances and records the transfer together.
- Notifications are created and the result is returned to the customer.

Complaint workflow

A customer selects an active admin and submits a complaint. The backend validates the selection, saves the assignment, and marks the complaint ASSIGNED. It appears in that admin's dashboard. Separate officer and manager screens support handling and escalation.

Supported statuses: OPEN, ASSIGNED, IN_PROGRESS, ESCALATED, RESOLVED, and CLOSED.

User deletion behavior

An admin confirms deletion from the user details page. Linked bank accounts become inactive while banking history is retained. Admin complaints assigned to the deleted user transfer to the deleting admin. These changes run in one database transaction.

Presentation summary

My project is a MERN-based banking simulation with four user roles. It combines account management, fund transfers, transaction tracking, complaint handling, and administration using authentication, role checks, validation, and database transactions.